Teste 1

```
import numpy as np
import pandas as pd
import scipy.stats as stats
import statsmodels.api as sm
from statsmodels.stats.multicomp import pairwise_tukeyhsd
```

```
df = pd.read_csv("/content/results_quarta_leva_3.csv")
df["modelo"].unique()
```

```
array(['Mistral3.2-24B', 'Mistral4-119B', 'Qwen3.6-35B', 'gemma-4-31B',
       'gpt-oss-120B'], dtype=object)
```

```
df
```

	modelo	quantizacao	total_smiles	total_smiles_validos	total_smiles_duplicados	token_per_second_medio	duration_min_media	% smiles validos
0	Mistral3.2-24B	BF16	1431	1395	54	2.324999	2.272222	97.484277
1	Mistral3.2-24B	Q6	1458	1341	18	42.867239	0.131711	91.975309
2	Mistral3.2-24B	Q8	1464	1422	24	39.990139	0.138919	97.131148
3	Mistral4-119B	BF16	858	742	14	4.433330	13.764699	86.480186
4	Mistral4-119B	Q4	782	546	42	14.895412	3.329497	69.820972
5	Mistral4-119B	Q6	834	726	10	9.710896	5.793336	87.050360
6	Mistral4-119B	Q8	874	764	6	7.922958	7.699337	87.414188
7	Qwen3.6-35B	BF16	1506	1446	0	13.921353	8.673914	96.015936
8	Qwen3.6-35B	Q4	1488	1401	9	98.857816	1.061600	94.153226
9	Qwen3.6-35B	Q6	1500	1467	6	98.120189	1.117502	97.800000
10	Qwen3.6-35B	Q8	1500	1422	0	81.941316	1.533501	94.800000
11	gemma-4-31B	BF16	1500	1338	0	0.989703	36.371630	89.200000
12	gemma-4-31B	Q4	1500	1392	0	34.798384	1.498846	92.800000
13	gemma-4-31B	Q6	1500	1356	0	26.787256	1.957950	90.400000
14	gemma-4-31B	Q8	1500	1338	0	25.974218	1.630863	89.200000
15	gpt-oss-120B	BF16	1500	1404	0	12.618265	9.851846	93.600000
16	gpt-oss-120B	Q4	1500	1461	0	16.220805	17.786117	97.400000
17	gpt-oss-120B	Q6	1500	1440	0	17.893738	8.233898	96.000000
18	gpt-oss-120B	Q8	1500	1392	0	15.037930	12.457462	92.800000

```
# 2. Cria a coluna de SMILES inválidos (essencial para a tabela de contingência)
df["total_smiles_invalidos"] = df["total_smiles"] - df["total_smiles_validos"]

# 3. Agrupa por modelo somando as contagens absolutas
tabela_contingente = df.groupby("modelo")[
    ["total_smiles_validos", "total_smiles_invalidos"]
].sum()

print("--- TABELA DE CONTINGÊNCIA CORRETA ---")
print(tabela_contingente)
print("\n" + "-" * 50 + "\n")
```

```
--- TABELA DE CONTINGÊNCIA CORRETA ---
           total_smiles_validos  total_smiles_invalidos
modelo
Mistral3.2-24B                 4158                     195
Mistral4-119B                  2778                     570
Qwen3.6-35B                    5736                     258
gemma-4-31B                    5424                     576
gpt-oss-120B                   5697                     303

--------------------------------------------------
```

```
chi2, p_val, dof, expected = stats.chi2_contingency(tabela_contingente)

if p_val < .05:
  print("Resultado: Significativo! Há diferença na taxa de validez entre modelos.")
  print(f"p-value: {p_val:.4e}\n")
else:
  print("Resultado: Não há diferença significativa na taxa de validez entre modelos.")
  print(f"p-value: {p_val:.4e}\n")
```

```
Resultado: Significativo! Há diferença na taxa de validez entre modelos.
p-value: 4.1196e-146
```

```
from itertools import combinations

pares = list(combinations(df["modelo"].unique(), 2))
n_comparacoes = len(pares)
alfa_corrigido = .05 / n_comparacoes
print(
    f"Número de comparações: {n_comparacoes} | Alfa corrigido de Bonferroni: {alfa_corrigido:.4f}\n"
)
```

```
Número de comparações: 10 | Alfa corrigido de Bonferroni: 0.0050
```

```
for p1, p2 in pares:
  df_par = df[df["modelo"].isin([p1, p2])]

  tab_par = df_par.groupby("modelo")[
      ["total_smiles_validos", "total_smiles_invalidos"]
  ].sum()

  _, p_par, _, _ = stats.chi2_contingency(tab_par)

  if p_par < alfa_corrigido:
    print(f"Resultado: Significativo! Há diferença na taxa de validez entre {p1} e {p2}.")
    print(f"p-value: {p_par:.4e}\n")
```

```
Resultado: Significativo! Há diferença na taxa de validez entre Mistral3.2-24B e Mistral4-119B.
p-value: 4.5187e-74

Resultado: Significativo! Há diferença na taxa de validez entre Mistral3.2-24B e gemma-4-31B.
p-value: 1.7046e-22

Resultado: Significativo! Há diferença na taxa de validez entre Mistral4-119B e Qwen3.6-35B.
p-value: 3.0240e-95

Resultado: Significativo! Há diferença na taxa de validez entre Mistral4-119B e gemma-4-31B.
p-value: 1.2912e-25

Resultado: Significativo! Há diferença na taxa de validez entre Mistral4-119B e gpt-oss-120B.
p-value: 7.8345e-81

Resultado: Significativo! Há diferença na taxa de validez entre Qwen3.6-35B e gemma-4-31B.
p-value: 6.2749e-30

Resultado: Significativo! Há diferença na taxa de validez entre gemma-4-31B e gpt-oss-120B.
p-value: 1.5727e-21
```

```
dados_gemma = df[df["modelo"] == "gemma-4-31B"]

gemma_q4 = dados_gemma[dados_gemma["quantizacao"] == "Q4"].iloc[0]
gemma_bf16 = dados_gemma[dados_gemma["quantizacao"] == "BF16"].iloc[0]

lista_sucessos = [
    gemma_q4["total_smiles_validos"],
    gemma_bf16["total_smiles_validos"],
]

lista_totais = [gemma_q4["total_smiles"], gemma_bf16["total_smiles"]]
print(f"Sucessos (Válidos) -> Q4: {lista_sucessos[0]} | BF16: {lista_sucessos[1]}")
print(f"Totais Gerados     -> Q4: {lista_totais[0]} | BF16: {lista_totais[1]}")
print("\n" + "-" * 50 + "\n")
```

```
Sucessos (Válidos) -> Q4: 1392 | BF16: 1338
Totais Gerados     -> Q4: 1500 | BF16: 1500

--------------------------------------------------
```

```
from statsmodels.stats.proportion import proportions_ztest

estatistica_z, p_valor = proportions_ztest(lista_sucessos, lista_totais)
print("--- RESULTADO DO TESTE Z DE PROPORÇÕES ---")
print(f"Estatística Z: {estatistica_z:.4f}")
print(f"p-valor: {p_valor:.6f}")

# 5. Tomada de decisão baseada no limiar de 5%
if p_valor < 0.05:
    print("\nResultado: ESTATISTICAMENTE SIGNIFICATIVO!")
    print(
        "A diferença é real! A compressão Q4 de fato gerou uma proporção "
        "de SMILES válidos estatisticamente superior à versão BF16."
    )
else:
    print("\nResultado: NÃO SIGNIFICATIVO.")
    print(
        "A diferença observada em porcentagem foi apenas uma oscilação ao acaso."
    )
```

```
--- RESULTADO DO TESTE Z DE PROPORÇÕES ---
Estatística Z: 3.4450
p-valor: 0.000571

Resultado: ESTATISTICAMENTE SIGNIFICATIVO!
A diferença é real! A compressão Q4 de fato gerou uma proporção de SMILES válidos estatisticamente superior à versão BF16.
```

```
df["modelo"].unique()
```

```
array(['Mistral3.2-24B', 'Mistral4-119B', 'Qwen3.6-35B', 'gemma-4-31B',
       'gpt-oss-120B'], dtype=object)
```

```
gpt_dados = df[df["modelo"] == "gpt-oss-120B"]
qwen_dados = df[df["modelo"] == "Qwen3.6-35B"]

gpt_quant_q4 = gpt_dados[gpt_dados["quantizacao"] == "Q4"].iloc[0]
gpt_quant_bf16 = gpt_dados[gpt_dados["quantizacao"] == "BF16"].iloc[0]
qwen_quant_q4 = qwen_dados[qwen_dados["quantizacao"] == "Q4"].iloc[0]
qwen_quant_bf16 = qwen_dados[qwen_dados["quantizacao"] == "BF16"].iloc[0]

lista_sucessos_gpt = [
    gpt_quant_q4["total_smiles_validos"],
    gpt_quant_bf16["total_smiles_validos"],
]
```

```
lista_sucessos_qwen = [
    qwen_quant_q4["total_smiles_validos"],
    qwen_quant_bf16["total_smiles_validos"],
]

lista_totais_gpt = [
    gpt_quant_q4["total_smiles"],
    gpt_quant_bf16["total_smiles"],
]

lista_totais_qwen = [
    qwen_quant_q4["total_smiles"],
    qwen_quant_bf16["total_smiles"],
]

print(f"Sucessos (Válidos) -> GPT Q4: {lista_sucessos_gpt[0]} | BF16: {lista_sucessos_gpt[1]}")
print(f"Totais Gerados     -> GPT Q4: {lista_totais_gpt[0]} | BF16: {lista_totais_gpt[1]}")
print("")
print(f"Sucessos (Válidos) -> Qwen Q4: {lista_sucessos_qwen[0]} | BF16: {lista_sucessos_qwen[1]}")
print(f"Totais Gerados     -> Qwen Q4: {lista_totais_qwen[0]} | BF16: {lista_totais_qwen[1]}")
print("\n" + "-" * 50 + "\n")
```

```
Sucessos (Válidos) -> GPT Q4: 1461 | BF16: 1404
Totais Gerados     -> GPT Q4: 1500 | BF16: 1500

Sucessos (Válidos) -> Qwen Q4: 1401 | BF16: 1446
Totais Gerados     -> Qwen Q4: 1488 | BF16: 1506

--------------------------------------------------
```

```
estatistica_z, p_valor = proportions_ztest(lista_sucessos_gpt, lista_totais_gpt)
print("--- RESULTADO DO TESTE Z DE PROPORÇÕES (GPT) ---")
print(f"Estatística Z: {estatistica_z:.4f}")
print(f"p-valor: {p_valor:.6f}")

if p_valor < .05:
  print("\nResultado: ESTATISTICAMENTE SIGNIFICATIVO!")
  print(
      "A diferença é real! Para o modelo GPT, a compressão Q4 de fato gerou uma proporção "
      "de SMILES válidos estatisticamente superior à versão BF16."
  )
else:
  print("\nResultado: NÃO SIGNIFICATIVO.")
  print(
      "A diferença observada em porcentagem para o modelo GPT foi apenas uma oscilação ao acaso."
  )
```

```
--- RESULTADO DO TESTE Z DE PROPORÇÕES (GPT) ---
Estatística Z: 5.0200
p-valor: 0.000001

Resultado: ESTATISTICAMENTE SIGNIFICATIVO!
A diferença é real! Para o modelo GPT, a compressão Q4 de fato gerou uma proporção de SMILES válidos estatisticamente superior à versão BF16.
```

```
estati_z, p_val = proportions_ztest(lista_sucessos_qwen, lista_totais_qwen)
print("--- RESULTADO DO TESTE Z DE PROPORÇÕES (Qwen) ---")
print(f"Estatística Z: {estati_z:.4f}")
print(f"p-valor: {p_val:.6f}")

if p_val < .05:
  print("\nResultado: ESTATISTICAMENTE SIGNIFICATIVO!")
  print(
      "A diferença é real! Para o modelo Qwen, a compressão Q4 de fato gerou uma proporção "
      "de SMILES válidos estatisticamente superior à versão BF16."
  )
else:
  print("\nResultado: NÃO SIGNIFICATIVO.")
  print(
      "A diferença observada em porcentagem para o modelo Qwen foi apenas uma oscilação ao acaso."
  )
```

```
--- RESULTADO DO TESTE Z DE PROPORÇÕES (Qwen) ---
Estatística Z: -2.3585
p-valor: 0.018350

Resultado: ESTATISTICAMENTE SIGNIFICATIVO!
A diferença é real! Para o modelo Qwen, a compressão Q4 de fato gerou uma proporção de SMILES válidos estatisticamente superior à versão BF16.
```

```
import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as stats

# 1. Definição dos parâmetros do seu teste real
# Graus de liberdade (gl) = (5 modelos - 1) * (2 categorias - 1) = 4
graus_de_liberdade = 4
alfa = 0.05  # Limiar de 5% da ciência

# Encontra o valor de corte matemático (Valor Crítico)
valor_critico = stats.chi2.ppf(1 - alfa, graus_de_liberdade)
seu_chi2 = 681.19

# 2. Criar os pontos no eixo X para desenhar a curva normal do acaso
# Vamos até 30 para conseguir enxergar o comportamento da curva
x = np.linspace(0, 30, 1000)
y = stats.chi2.pdf(x, graus_de_liberdade)

# 3. Iniciar o plot
plt.figure(figsize=(11, 6))
plt.plot(
    x, y, label=f"Distribuição $\chi^2$ (gl={graus_de_liberdade})", color="#1f77b4", lw=2.5
)

# 4. Pintar a Região de Rejeição (o limiar do "falso positivo")
x_critico = np.linspace(valor_critico, 30, 500)
y_critico = stats.chi2.pdf(x_critico, graus_de_liberdade)
plt.fill_between(
    x_critico,
    y_critico,
    color="#d62728",
    alpha=0.3,
    label=f"Região de Rejeição (Alfa = {alfa})",
)

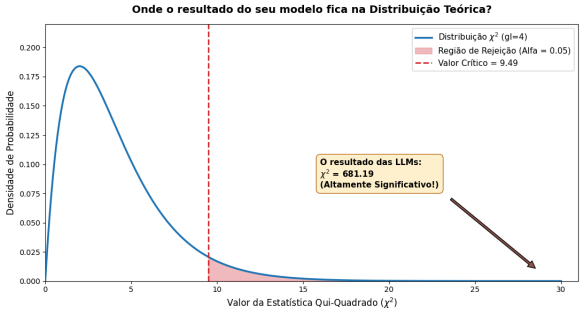
# Desenhar a linha vertical do corte
plt.axvline(
    valor_critico,
    color="#d62728",
    linestyle="--",
    lw=2,
    label=f"Valor Crítico = {valor_critico:.2f}",
)

# 5. Criar a seta apontando para o seu resultado real (que está fora do gráfico)
plt.annotate(
    f"O resultado das LLMs:\n$\chi^2$ = {seu_chi2}\n(Altamente Significativo!)",
    xy=(29, 0.005),  # Onde a seta vai apontar (borda do gráfico)
    xytext=(16, 0.08),  # Onde o balão de texto vai ficar centralizado
    arrowprops=dict(
        facecolor="#8c564b", shrink=0.08, width=3, headwidth=10, headlength=10
    ),
    fontsize=11,
    weight="bold",
    bbox=dict(boxstyle="round,pad=0.5", fc="#fff2cc", ec="#b45f06", alpha=0.9),
)

# 6. Detalhes estéticos
plt.title(
    "Onde o resultado do seu modelo fica na Distribuição Teórica?",
    fontsize=14,
    weight="bold",
    pad=15,
)
plt.xlabel("Valor da Estatística Qui-Quadrado ($\chi^2$)", fontsize=12)
plt.ylabel("Densidade de Probabilidade", fontsize=12)
plt.xlim(0, 31)
plt.ylim(0, 0.22)
plt.legend(loc="upper right", frameon=True, fontsize=11)
plt.tight_layout()

# Exibir no notebook
plt.show()
```

```
<>:22: SyntaxWarning: invalid escape sequence '\c'
<>:47: SyntaxWarning: invalid escape sequence '\c'
<>:65: SyntaxWarning: invalid escape sequence '\c'
<>:22: SyntaxWarning: invalid escape sequence '\c'
<>:47: SyntaxWarning: invalid escape sequence '\c'
<>:65: SyntaxWarning: invalid escape sequence '\c'
/tmp/ipykernel_955/7295669.py:22: SyntaxWarning: invalid escape sequence '\c'
  x, y, label=f"Distribuição $\chi^2$ (gl={graus_de_liberdade})", color="#1f77b4", lw=2.5
/tmp/ipykernel_955/7295669.py:47: SyntaxWarning: invalid escape sequence '\c'
  f"O resultado das LLMs:\n$\chi^2$ = {seu_chi2}\n(Altamente Significativo!)",
/tmp/ipykernel_955/7295669.py:65: SyntaxWarning: invalid escape sequence '\c'
  plt.xlabel("Valor da Estatística Qui-Quadrado ($\chi^2$)", fontsize=12)
```




```
df.head()
```

	modelo	quantizacao	total_smiles	total_smiles_validos	total_smiles_duplicados	token_per_second_medio	duration_min_media	% smiles validos	total_smiles_invalidos
0	Mistral3.2-24B	BF16	1431	1395	54	2.324999	2.272222	97.484277	36
1	Mistral3.2-24B	Q6	1458	1341	18	42.867239	0.131711	91.975309	117
2	Mistral3.2-24B	Q8	1464	1422	24	39.990139	0.138919	97.131148	42
3	Mistral4-119B	BF16	858	742	14	4.433330	13.764699	86.480186	116
4	Mistral4-119B	Q4	782	546	42	14.895412	3.329497	69.820972	236

```
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
import statsmodels.api as sm
from statsmodels.formula.api import ols

# 1. Carregar o teu arquivo de dados reais
df = pd.read_csv("results_quarta_leva_3.csv")

# 2. Executar a ANOVA de Duas Vias (Two-Way ANOVA)
# A fórmula diz: "Velocidade (token_per_second_medio) explicada pelo modelo + quantizacao"
modelo_linear = ols(
    "token_per_second_medio ~ C(modelo) + C(quantizacao)", data=df
).fit()
tabela_anova = sm.stats.anova_lm(modelo_linear, typ=2)

# 3. Formatar e exibir os resultados estatísticos de forma limpa
print("=" * 60)
print("       RESULTADO DA ANOVA DE DUAS VIAS (TOKENS/S)       ")
print("=" * 60)
print(tabela_anova)
print("=" * 60)

# Extrair os p-valores para criar uma lógica de decisão automática
p_modelo = tabela_anova.loc["C(modelo)", "PR(>F)"]
p_quant = tabela_anova.loc["C(quantizacao)", "PR(>F)"]

print("\n--- ANÁLISE DO VEREDITO MATEMÁTICO ---")
if p_modelo < 0.05:
    print(
        f"-> O fator MODELO é SIGNIFICATIVO (p = {p_modelo:.4e}).\n"
        "   Mesmo com as oscilações de quantização, existe uma diferença brutal e real "
        "   na velocidade de geração entre os modelos, exatamente como o teu gráfico aponta!"
    )
else:
    print(
        f"-> O fator MODELO NÃO é significativo (p = {p_modelo:.4f})."
    )

if p_quant < 0.05:
    print(
        f"\n-> O fator QUANTIZAÇÃO é SIGNIFICATIVO (p = {p_quant:.4e}).\n"
        "   Mudar o nível de compressão (ex: BF16 para Q4) altera significativamente "
        "   a velocidade dos tokens por segundo."
    )
else:
    print(
        f"\n-> O fator QUANTIZAÇÃO NÃO é significativo (p = {p_quant:.4f})."
    )

print("\n" + "=" * 60 + "\n")

# 4. Gerar um gráfico auxiliar descritivo (Boxplot) para complementar o teu relatório
sns.set_theme(style="whitegrid")
plt.figure(figsize=(12, 6))

sns.boxplot(
    x="modelo",
    y="token_per_second_medio",
    data=df,
    hue="modelo",
    palette="Set2",
    legend=False,
)
# Opcional: Adiciona os pontos reais (cada quantização) por cima do boxplot para transparência total
sns.stripplot(
    x="modelo", y="token_per_second_medio", data=df, color="black", size=6, jitter=0.1
)

plt.title(
    "Variabilidade da Velocidade de Geração por Modelo (Todas as Quantizações)",
    fontsize=14,
    weight="bold",
    pad=15,
)
plt.xlabel("Modelos Analisados", fontsize=12)
plt.ylabel("Tokens por Segundo Médio", fontsize=12)
plt.xticks(rotation=15, ha="right")
plt.tight_layout()

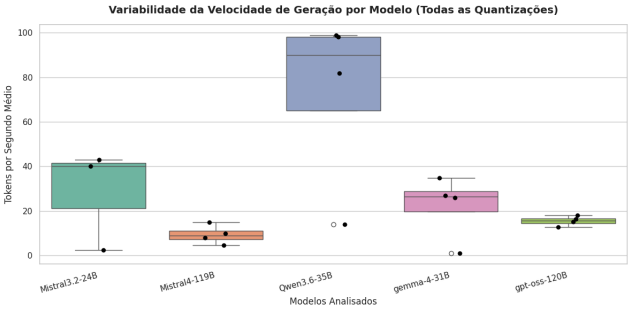
# Exibe o gráfico no Colab
plt.show()
```

```
============================================================
       RESULTADO DA ANOVA DE DUAS VIAS (TOKENS/S)
============================================================
                    sum_sq    df    F    PR(>F)
C(modelo)       10297.954891   4.0  9.672038  0.001322
C(quantizacao)   3680.947997   3.0  4.609623  0.025322
Residual         2927.963812  11.0       NaN       NaN
============================================================

--- ANÁLISE DO VEREDITO MATEMÁTICO ---
-> O fator MODELO é SIGNIFICATIVO (p = 1.3216e-03).
   Mesmo com as oscilações de quantização, existe uma diferença brutal e real    na velocidade de geração entre os modelos, exatamente como o teu gráfico aponta!

-> O fator QUANTIZAÇÃO é SIGNIFICATIVO (p = 2.5322e-02).
   Mudar o nível de compressão (ex: BF16 para Q4) altera significativamente    a velocidade dos tokens por segundo.

============================================================
```




```
df.columns
```

```
Index(['modelo', 'quantizacao', 'total_smiles', 'total_smiles_validos',
       'total_smiles_duplicados', 'token_per_second_medio',
       'duration_min_media', '% smiles validos'],
      dtype='object')
```

```
import pandas as pd
import scipy.stats as stats

# 1. Carrega os dados
df = pd.read_csv("results_quarta_leva_3.csv")

# 2. Transforma os dados em uma matriz (Linhas = Quantizações, Colunas = Modelos)
matriz_tempo = df.pivot(
    index="quantizacao", columns="modelo", values="duration_min_media"
)

# Como falta 1 linha para completar as 20 combinações, preenchemos com a média do modelo
matriz_tempo = matriz_tempo.fillna(matriz_tempo.mean())

print("--- MATRIZ DE CRUZA DE TEMPOS ---")
print(matriz_tempo)
print("\n" + "-" * 50 + "\n")

# 3. Executa o Teste de Friedman
stat_f, p_val_f = stats.friedmanchisquare(
    *[matriz_tempo[col] for col in matriz_tempo.columns]
)

print("--- RESULTADO DO TESTE DE FRIEDMAN ---")
print(f"Estatística de Friedman: {stat_f:.4f}")
print(f"p-valor: {p_val_f:.5f}")

if p_val_f < 0.05:
    print(
        "\nResultado: ESTATISTICAMENTE SIGNIFICATIVO!"
    )
    print(
        "O teste não-paramétrico confirmou que a diferença de tempo entre os modelos é real!"
    )
```

```
--- MATRIZ DE CRUZA DE TEMPOS ---
modelo       Mistral3.2-24B  Mistral4-119B  Qwen3.6-35B  gemma-4-31B  \
quantizacao
BF16               2.272222      13.764699     8.673914    36.371630
Q4                 0.847617       3.329497     1.061600     1.498846
Q6                 0.131711       5.793336     1.117502     1.957950
Q8                 0.138919       7.699337     1.533501     1.630863

modelo       gpt-oss-120B
quantizacao
BF16             9.851846
Q4              17.786117
Q6               8.233898
Q8              12.457462

--------------------------------------------------

--- RESULTADO DO TESTE DE FRIEDMAN ---
Estatística de Friedman: 13.6000
p-valor: 0.00869
```

```
Resultado: ESTATISTICAMENTE SIGNIFICATIVO!
O teste não-paramétrico confirmou que a diferença de tempo entre os modelos é real!
```

```
print("--- Pergunta 5: Correlação Biserial Ponto ---")

corr, p_corr = stats.pointbiserialr(df["total_smiles_validos"], df["token_per_second_medio"])

print(f"Coeficiente de Correlação: {corr:.4f}")
print(f"p-valor da Correlação: {p_corr:.4e}")

if p_corr < 0.05:
    if corr > 0:
        print(
            "Resultado: Correlação positiva significativa! Modelos mais rápidos geram mais SMILES válidos."
        )
    else:
        print(
            "Resultado: Correlação negativa significativa! Modelos mais rápidos tendem a errar mais a sintaxe."
        )
else:
    print(
        "Resultado: Não há correlação estatisticamente significativa entre velocidade e validez."
    )
print("-" * 50)
```

```
--- Pergunta 5: Correlação Biserial Ponto ---
Coeficiente de Correlação: 0.3748
p-valor da Correlação: 1.1388e-01
Resultado: Não há correlação estatisticamente significativa entre velocidade e validez.
--------------------------------------------------
```

Novo teste

```
pip install scikit-posthocs
```

```
import pandas as pd
import numpy as np
from scipy import stats
from scipy.stats import shapiro, levene, kruskal, mannwhitneyu, spearmanr
import statsmodels.api as sm
from statsmodels.formula.api import glm
from statsmodels.stats.multicomp import pairwise_tukeyhsd
import scikit_posthocs as sp
import warnings
warnings.filterwarnings("ignore")
```

```
df = pd.read_csv("/content/results_quarta_leva_2.csv")
```

```
# Remove réplicas com 0 SMILES gerados (falhas de execução)
df_clean = df[df["total_smiles"] > 0].copy()
df_clean["taxa_validos"] = df_clean["total_smiles_validos"] / df_clean["total_smiles"]
df_clean["grupo"] = df_clean["modelo"] + "_" + df_clean["quantizacao"]
df_clean = df_clean[df_clean["total_smiles"] >= 10].copy()
df_clean = df_clean[~df_clean["modelo"].str.contains("Mistral3.2-24B", case=False)]

df_clean.head()
```

	leva	modelo	quantizacao	replica	total_smiles	total_smiles_validos	total_smiles_duplicados	pct_validos	pct_invalidos	duration_min_media	token_per_second_medio	taxa_validos	grupo
0	quarta_leva	gemma-4-31B	Q8	gemma-4-31B-Q8_replica_8	20	20	0	100.0	0.0	1.630863	25.974218	1.00	gemma-4-31B_Q8
1	quarta_leva	gemma-4-31B	Q8	gemma-4-31B-Q8_replica_10	20	14	0	70.0	30.0	1.630863	25.974218	0.70	gemma-4-31B_Q8
3	quarta_leva	Qwen3.6-35B	Q6	Qwen3.6-35B-Q6_replica_11	20	19	0	95.0	5.0	1.117502	98.120189	0.95	Qwen3.6-35B_Q6
4	quarta_leva	gemma-4-31B	Q4	gemma-4-31B-Q4_replica_24	20	20	0	100.0	0.0	1.498846	34.798384	1.00	gemma-4-31B_Q4
6	quarta_leva	gpt-oss-120B	BF16	gpt-oss-120B-BF16_replica_6	20	19	0	95.0	5.0	9.851846	12.618265	0.95	gpt-oss-120B_BF16

```
# ETAPA 0: CARREGANDO E PREPARANDO

print("=== ANOMALIZAS DETECTADAS ===")
anomalias = df[df["total_smiles"] < 10]

print(anomalias[["modelo", "quantizacao", "replica", "total_smiles", "total_smiles_validos"]])
```

```
     modelo quantizacao                replica  total_smiles  total_smiles_validos
649   Mistral4-119B           Q6       Mistral4-119B-Q6_replica_6
685   Mistral4-119B           Q4     Mistral4-119B-Q4_replica_11
738   Mistral4-119B           Q4     Mistral4-119B-Q4_replica_16
743  Mistral3.2-24B        BF16  Mistral3.2-24B-BF16_replica_17
762   Mistral4-119B           Q4     Mistral4-119B-Q4_replica_19
808   Mistral4-119B           Q8     Mistral4-119B-Q8_replica_20
869   Mistral4-119B        BF16   Mistral4-119B-BF16_replica_15
878   Mistral4-119B           Q4      Mistral4-119B-Q4_replica_9
880   Mistral4-119B           Q8     Mistral4-119B-Q8_replica_10
888   Mistral4-119B           Q6      Mistral4-119B-Q6_replica_5
896   Mistral4-119B           Q6     Mistral4-119B-Q6_replica_23
898   Mistral4-119B           Q8      Mistral4-119B-Q8_replica_2
929   Mistral4-119B           Q4     Mistral4-119B-Q4_replica_24
993   Mistral4-119B        BF16   Mistral4-119B-BF16_replica_24
1017  Mistral4-119B           Q4      Mistral4-119B-Q4_replica_4
1032  Mistral4-119B           Q4      Mistral4-119B-Q4_replica_3
1045  Mistral4-119B        BF16    Mistral4-119B-BF16_replica_7
1054  Mistral4-119B           Q4     Mistral4-119B-Q4_replica_22
1092  Mistral4-119B           Q6     Mistral4-119B-Q6_replica_21
1124  Mistral4-119B           Q6      Mistral4-119B-Q6_replica_6
1160  Mistral4-119B           Q4     Mistral4-119B-Q4_replica_11
1213  Mistral4-119B           Q4     Mistral4-119B-Q4_replica_16
1218 Mistral3.2-24B        BF16  Mistral3.2-24B-BF16_replica_17
1237  Mistral4-119B           Q4     Mistral4-119B-Q4_replica_19
1283  Mistral4-119B           Q8     Mistral4-119B-Q8_replica_20

      total_smiles  total_smiles_validos
288              2                     0
394              0                     0
403              0                     0
405              1                     1
413              0                     0
421              0                     0
423              0                     0
454              0                     0
518              0                     0
542              1                     1
557              2                     1
570              0                     0
579              0                     0
617              0                     0
649              0                     0
685              2                     1
738              0                     0
743              2                     0
762              3                     1
808              0                     0
869              0                     0
878              0                     0
880              1                     1
888              0                     0
896              0                     0
898              0                     0
929              0                     0
993              0                     0
1017             1                     1
1032             2                     1
1045             0                     0
```

```
df_clean.head()
```

	leva	modelo	quantizacao	replica	total_smiles	total_smiles_validos	total_smiles_duplicados	pct_validos	pct_invalidos	duration_min_media	token_per_second_medio	taxa_validos	grupo
0	quarta_leva	gemma-4-31B	Q8	gemma-4-31B-Q8_replica_8	20	20	0	100.0	0.0	1.630863	25.974218	1.00	gemma-4-31B_Q8
1	quarta_leva	gemma-4-31B	Q8	gemma-4-31B-Q8_replica_10	20	14	0	70.0	30.0	1.630863	25.974218	0.70	gemma-4-31B_Q8
3	quarta_leva	Qwen3.6-35B	Q6	Qwen3.6-35B-Q6_replica_11	20	19	0	95.0	5.0	1.117502	98.120189	0.95	Qwen3.6-35B_Q6
4	quarta_leva	gemma-4-31B	Q4	gemma-4-31B-Q4_replica_24	20	20	0	100.0	0.0	1.498846	34.798384	1.00	gemma-4-31B_Q4
6	quarta_leva	gpt-oss-120B	BF16	gpt-oss-120B-BF16_replica_6	20	19	0	95.0	5.0	9.851846	12.618265	0.95	gpt-oss-120B_BF16

```
desc = df_clean.groupby(["modelo", "quantizacao"])["taxa_validos"].agg(
    n="count",
    media="mean",
    mediana="median",
    desvio_padrao="std",
    IQR=lambda x: x.quantile(.75) - x.quantile(.25),
    minimo="min",
    maximo="max",
    CV=lambda x: x.std() / x.mean() * 100 if x.mean() > 0 else np.nan
).round(4)

print("\n=== ESTATÍSTICA DESCRITIVA POR GRUPO ===")
print(desc.to_string())
```

```
=== ESTATÍSTICA DESCRITIVA POR GRUPO ===
                          n   media  mediana  desvio_padrao     IQR  minimo  maximo       CV
modelo        quantizacao
Mistral4-119B BF16       44  0.8644   0.8461         0.0871  0.1500  0.7000     1.0  10.0790
              Q4         34  0.7409   0.7917         0.2845  0.1700  0.0000     1.0  38.4032
              Q6         42  0.8710   0.8947         0.0997  0.1722  0.7000     1.0  11.4462
              Q8         44  0.8742   0.9000         0.1492  0.1818  0.4500     1.0  17.0629
Qwen3.6-35B   BF16       75  0.9607   1.0000         0.0531  0.0500  0.8182     1.0   5.5250
              Q4         75  0.9385   1.0000         0.0980  0.1000  0.5625     1.0  10.4411
              Q6         75  0.9780   1.0000         0.0351  0.0500  0.9000     1.0   3.5896
              Q8         75  0.9480   0.9500         0.0595  0.1000  0.8000     1.0   6.2790
gemma-4-31B   BF16       75  0.8920   0.9000         0.1014  0.1000  0.5000     1.0  11.3627
              Q4         75  0.9280   0.9500         0.0997  0.1000  0.6000     1.0  10.7482
              Q6         75  0.9040   0.9000         0.0651  0.1000  0.8000     1.0   7.2035
              Q8         75  0.8920   0.9500         0.1014  0.1000  0.6500     1.0  11.3627
gpt-oss-120B  BF16       75  0.9360   0.9500         0.0645  0.1000  0.7500     1.0   6.8904
              Q4         75  0.9740   1.0000         0.0289  0.0500  0.9000     1.0   2.9670
              Q6         75  0.9600   0.9500         0.0450  0.0500  0.8500     1.0   4.6898
              Q8         75  0.9280   0.9500         0.0831  0.1000  0.6500     1.0   8.9564
```

```
# ETAPA 2: VERIFCAÇÃO DE PREMISSAS

# 2. Normalização
print("=== TESTE DE NORMALIZAÇÃO (Shapiro-Wilk) ===")

normalizacao_ok = True

for grupo, dados in df_clean.groupby("grupo")["taxa_validos"]:
  dados_validos = dados.dropna()

  if len(dados_validos) >= 3:
    stat, p = shapiro(dados_validos)

    resultado = "Normal" if p > .05 else "Não-normal"

    if p <= .05:
      normalizacao_ok = False
    print(f"Grupo: {grupo} | Estatística: {stat:.4f} | p-valor: {p:.4f} | Resultado: {resultado}")
```

```
=== TESTE DE NORMALIZAÇÃO (Shapiro-Wilk) ===
Grupo: Mistral4-119B_BF16 | Estatística: 0.9346 | p-valor: 0.0151 | Resultado: Não-normal
Grupo: Mistral4-119B_Q4 | Estatística: 0.6722 | p-valor: 0.0000 | Resultado: Não-normal
Grupo: Mistral4-119B_Q6 | Estatística: 0.8986 | p-valor: 0.0013 | Resultado: Não-normal
Grupo: Mistral4-119B_Q8 | Estatística: 0.7978 | p-valor: 0.0000 | Resultado: Não-normal
```

```
Grupo: Qwen3.6-35B_BF16 | Estatística: 0.7436 | p-valor: 0.0000 | Resultado: Não-normal
Grupo: Qwen3.6-35B_Q4 | Estatística: 0.6552 | p-valor: 0.0000 | Resultado: Não-normal
Grupo: Qwen3.6-35B_Q6 | Estatística: 0.6395 | p-valor: 0.0000 | Resultado: Não-normal
Grupo: Qwen3.6-35B_Q8 | Estatística: 0.8050 | p-valor: 0.0000 | Resultado: Não-normal
Grupo: gemma-4-31B_BF16 | Estatística: 0.7541 | p-valor: 0.0000 | Resultado: Não-normal
Grupo: gemma-4-31B_Q4 | Estatística: 0.7139 | p-valor: 0.0000 | Resultado: Não-normal
Grupo: gemma-4-31B_Q6 | Estatística: 0.9014 | p-valor: 0.0000 | Resultado: Não-normal
Grupo: gemma-4-31B_Q8 | Estatística: 0.8556 | p-valor: 0.0000 | Resultado: Não-normal
Grupo: gpt-oss-120B_BF16 | Estatística: 0.8318 | p-valor: 0.0000 | Resultado: Não-normal
Grupo: gpt-oss-120B_Q4 | Estatística: 0.7140 | p-valor: 0.0000 | Resultado: Não-normal
Grupo: gpt-oss-120B_Q6 | Estatística: 0.7919 | p-valor: 0.0000 | Resultado: Não-normal
Grupo: gpt-oss-120B_Q8 | Estatística: 0.7905 | p-valor: 0.0000 | Resultado: Não-normal
```

```
# 2b . Homogeneidade de variancias
grupos_dados = [g["taxa_validos"] for _, g in df_clean.groupby("grupo")]
stat_lev, p_lev = levene(*grupos_dados, center="median")

print(f"\n=== TESTE DE LEVENE (Brown-Forsythe) ===")
print(f"  Estatística={stat_lev:.4f} | p={p_lev:.4f}")
print(f"  {'✅ Variâncias homogêneas' if p_lev > 0.05 else '❌ Variâncias heterogêneas'}")
```

```
=== TESTE DE LEVENE (Brown-Forsythe) ===
  Estatística=9.7840 | p=0.0000
  ❌ Variâncias heterogêneas
```

```
# TESTE 3: TESTES DE HIPOTESES


# --- Caminho A: Não-Paramétrico ---
# H1: Efeito do modelo
print("\n=== KRUSKAL-WALLIS: Efeito do Modelo ===")

grupos_modelo = [g["taxa_validos"] for _, g in df_clean.groupby("modelo")]
stat_kw, p_kw = kruskal(*grupos_modelo)

print(f" H={stat_kw:.4f} | p={p_kw:.4e}")

if p_kw < .05:
  print("\nDiferença significativa! Aplicando post-hoc Dunn:\n")
  dunn = sp.posthoc_dunn(df_clean, val_col="taxa_validos", group_col="modelo")

  print(dunn.round(4).to_string())
```

```
=== KRUSKAL-WALLIS: Efeito do Modelo ===
 H=162.6201 | p=4.9851e-35

Diferença significativa! Aplicando post-hoc Dunn:

             Mistral4-119B  Qwen3.6-35B  gemma-4-31B  gpt-oss-120B
Mistral4-119B           1.0       0.0000          0.0        0.0000
Qwen3.6-35B             0.0       1.0000          0.0        0.0587
gemma-4-31B             0.0       0.0000          1.0        0.0000
gpt-oss-120B            0.0       0.0587          0.0        1.0000
```

```
#H2: Efeito da quantização
print("\n=== KRUSKAL-WALLIS: Efeito da Quantização ===")

grupos_quant = [g["taxa_validos"] for _, g in df_clean.groupby("quantizacao")]
stat_kq, p_kq = kruskal(*grupos_quant)
print(f" H={stat_kq:.4f} | p={p_kq:.4e}")
```

```
=== KRUSKAL-WALLIS: Efeito da Quantização ===
 H=11.9980 | p=7.3900e-03
```

```
# ============================================================
# ETAPA 4: CORRELAÇÕES (nível de grupo)
# ============================================================
print("\n=== CORRELAÇÕES DE GRUPO ===")
grupo_stats = df_clean.groupby('grupo').agg(
    taxa_media=('taxa_validos', 'mean'),
    token_s=('token_per_second_medio', 'first'),
    duracao=('duration_min_media', 'first')
).reset_index()

rho_tok, p_tok = spearmanr(grupo_stats['token_s'], grupo_stats['taxa_media'])
rho_dur, p_dur = spearmanr(grupo_stats['duracao'], grupo_stats['taxa_media'])
print(f"  Token/s vs Taxa válidos: ρ={rho_tok:.4f} | p={p_tok:.4f}")
print(f"  Duração vs Taxa válidos: ρ={rho_dur:.4f} | p={p_dur:.4f}")
```

```
=== CORRELAÇÕES DE GRUPO ===
  Token/s vs Taxa válidos: ρ=0.5611 | p=0.0237
  Duração vs Taxa válidos: ρ=-0.1399 | p=0.6053
```

Mapa de calor para verificar a diferença entre as médias de cada quantização

```
import seaborn as sns
import matplotlib.pyplot as plt

pivot = df_clean.pivot_table(values="taxa_validos", index="modelo", columns="quantizacao", aggfunc="mean")
pivot_pct = pivot * 100
plt.figure(figsize=(8, 6))
sns.heatmap(pivot_pct, annot=True, fmt=".2f", cmap="RdYlGn", vmin=70, vmax=100)
plt.title("Taxa de Validação (%) por Modelo e Quantização")
plt.show()
```

